Business Sovereignty through Replaceable Implementations

An Exploratory Study of AI Assisted Software Development

Keiji Jin

Research manuscript draft 1.1 • 7 November 2026 • Not peer reviewed

Abstract

AI coding assistants make alternative implementations easier to produce, but generating another codebase does not establish that an organization can safely replace its system. We propose Business Sovereignty Development, a method that treats business intent, an explicit exchange contract, and versioned evidence as durable assets, and treats implementation replacement as a conditional engineering decision. We investigate the method through a constructed design-change approval application with separately implemented JavaScript and Python APIs. A supplied review identified contract discrepancies despite an earlier record of 21 passing acceptance checks per implementation: Unicode length, timestamp parsing, and Boolean–integer equality differed across languages. It also identified a local storage transport limit below the declared record capacity. We refined the contract and implementations and analyzed the recorded evaluation. Both implementations passed the same 44 acceptance checks. Two small migration fixtures, one per direction, satisfied six assertions concerning import success, request preservation, and policy-audit preservation. Additional checks exercised 200 maximum-field-length Unicode records per source implementation and migration into the other implementation. These observations support bounded feasibility and expose conditions that ordinary acceptance counts can miss. They do not establish complete behavioral equivalence, autonomous regeneration, lower development cost, or independence from AI providers. The contribution is an operational replacement criterion, a concrete method for returning counterexamples to the specification, and an inspectable exploratory artifact.

Keywords: software evolution; specification-driven development; implementation replacement; AI-assisted software engineering; data migration; test oracles; audit preservation.

1 Introduction

A software system can remain technically maintainable while an organization loses practical control over its business rules. Rules may be dispersed across source files, deployment scripts, storage conventions, and informal conversations. Replacing the implementation then requires recovering both the meaning of the business and the accumulated record of decisions. Source-code availability alone does not answer whether another implementation will interpret the same request, preserve the same responsibilities, or retain the same history.

This problem belongs to an established engineering lineage. System/360 deliberately provided compatibility across machines [1]. Data independence sought to insulate applications from storage representation [2], and information hiding separated volatile design decisions from their clients [3]. These precedents motivate a further question: what evidence would permit an organization to choose a different implementation while retaining authority over its business commitments? They do not justify a historical claim that one computer architecture caused organizational dependence, or that a subsequent technology necessarily abolishes it.

AI-assisted development makes this question timely because code generation and repository modification are active research tasks [9,10]. Yet neither producing code nor repairing an issue, by itself, demonstrates

business-system replaceability. A replacement can pass familiar examples and still reject valid international text, accept a malformed history, or fail before reaching its documented data capacity. The cost of creating an alternative and the justification for adopting it are different research questions.

We use **business sovereignty** to denote a proposed engineering objective: an organization's practical ability to define and approve its business commitments, retain the data and history that embody them, and select an implementation on inspectable evidence. The term does not assert legal sovereignty, complete technological independence, or an empirically established organizational outcome. The present study examines technical enabling conditions in a small constructed case.

The paper contributes three elements. First, it defines a conditional replacement criterion combining behavioral checks, state and history preservation, operational limits, and authority boundaries. Second, it describes a specification-centered development and review cycle in which counterexamples update both the contract and its checks. Third, it reports an artifact-based investigation of two realizations and explains precisely what the observations do and do not support. We do not claim to invent specification-first development, contracts, or portability.

2 Background and Related Work

2.1 Abstraction and preservation of intent

Parnas's decomposition criterion places changeable design decisions behind module boundaries [3]. Design by Contract makes obligations explicit [4], while behavioral subtyping formulates conditions for substitutability at the type level [5]. These ideas inform our replacement criterion; our finite API checks do not prove behavioral subtyping or refinement. Leveson's intent specifications address human reasoning about why a system is designed as it is [6]. We similarly retain business rationale, while adding a concrete migration and evidence workflow for this artifact.

The OMG's Model Driven Architecture separates business functionality from platform technology [7]. Whittle and colleagues' industrial study reports selective rather than universal use of model-driven generation [8]. Our artifact shares the separation objective, but uses a natural-language method, JSON configuration, HTTP checks, and independent handlers rather than an established model-transformation language. Its JSON is not a complete executable semantics or a demonstrated universal generator.

2.2 AI assisted and specification driven development

HumanEval evaluates functional program synthesis from descriptions [9], and SWE-bench evaluates changes to existing repositories in response to issues [10]. Their units of evaluation differ from our stateful replacement task. We make no numerical comparison with either benchmark.

Recent specification-driven development literature overlaps directly with this proposal. Piskala distinguishes several levels of specification rigor [11]. Diaz and colleagues analyze specifications as part of the governance of human–agent collaboration [12]. Agarwal and colleagues examine prescribed and actual practices, including the tension between intent and implementation detail [13]. We cite versioned preprints as emerging related work, not as established evidence of universal effectiveness. Our narrower contribution is an explicit replacement gate and a worked investigation of cross-language contract failures, migration, and operational boundaries. A targeted literature search cannot establish that this combination is unprecedented.

2.3 Oracles and empirical interpretation

A test needs an oracle for distinguishing required behavior from observed behavior [14]. Agreement between implementations can reveal discrepancies, but cannot rule out a shared misunderstanding. Knight and Leveson's multiversion study warns against assuming independent failures merely because versions are developed separately [15]. Here both handlers and their tests were produced in the same AI-assisted work context, so correlated error is an especially important threat.

Runeson and Höst emphasize context, evidence, and limitations in software-engineering case reporting [16]. We adopt that reporting discipline while describing our setting accurately: this is a constructed technical case and retrospective artifact analysis, not an industrial field study or a controlled comparison of development methods.

3 Business Sovereignty Development

3.1 Durable assets and the replacement gate

The method retains three kinds of asset. Business intent records purposes, responsibilities, and exceptions in language that stakeholders can inspect. The contract specifies observable operations, representations, authorization, retained history, concurrency, and capacity. Evidence records the inputs and versions actually examined, their outcomes, and unperformed evaluations. Implementation source remains an important asset, but is not the sole authority on business meaning.

We define a **replacement gate** as a decision requiring five documented conditions for a stated workload and observation boundary. The source and candidate must satisfy the selected contract checks. Migration must preserve the declared business-state projection. Retained audit history must remain interpretable under its recorded policy versions. The candidate must support the declared operational envelope without hidden lower limits. Finally, export, import, and policy change must respect explicitly assigned authority. Passing this gate is relative to the checks and assumptions disclosed; it is not a proof for every input or future environment.

The preservation projection in this artifact concerns request identifiers, attributes, status, creation information, per-request histories, policy version, and policy-change history. Workspace-specific authentication identifiers, learning progress, implementation-specific revision counters, and export timestamps are outside the business migration claim. The runner directly compares request arrays and policy-history arrays; it does not independently assert equality of every top-level export field. Import assigns and validates policy version through implementation logic. This distinction limits the strength of the recorded preservation claim.

3.2 Development and review cycle

The cycle begins by collecting business purposes and counterexamples. It then specifies states, roles, wire representations, retention, conflicts, and limits. Public-interface checks are prepared separately from handlers. AI assistance is used to implement and revise the system, with human responsibility for adopting rules and judging evidence. A second realization and migrations exercise the same commitments. Discovered discrepancies return to the specification, checks, implementation, and explanatory material together.

This cycle permits revision of the contract. It does not freeze the initial test suite and subsequently present all added checks as an independent holdout. In the present case, the review influenced both the specification and the final 44-check suite. The observed result is therefore conformance after iterative repair, not a blind estimate of generalization from an untouched evaluation set.

3.3 Role of AI

AI assistance supported drafting, coding, investigation, and remediation. The retained artifact does not provide a frozen generation protocol, verified model identifiers for every implementation step, generation seeds, comparable token budgets, or a complete intervention log. Consequently, we evaluate the resulting engineering artifact and method, not a particular model's ability to regenerate it. Independence refers only to separately written language-specific handlers, not independent teams, independent model providers, or absence of source exposure.

4 Research Design

4.1 Questions and unit of analysis

RQ1 asks whether two separately implemented APIs can satisfy the refined observable business contract. RQ2 asks whether selected business state and histories survive migration in both directions and at the record-count boundary. RQ3 asks which discrepancies in this development episode escaped the earlier shared acceptance suite and how they inform replacement criteria.

The unit of analysis is one synthetic design-change approval application and its revision episode. The two language realizations are not two independent organizational cases. Recorded assertion groups are not statistical samples. We report descriptive outcomes and analytical implications, without significance tests, confidence intervals, causal effect estimates, or a combined success percentage.

4.2 Artifact and environment

The application supports draft creation, correction, submission, approval, rejection with a reason, and withdrawal. Three synthetic personas share a management tenant: a designer, an approver, and a combined designer–approver. A fourth persona belongs to another tenant. Policy v1 allows self-approval; v2 rejects self-approval and self-rejection. Denied writes must preserve business data, history, and revision.

The JavaScript realization runs locally on Node.js 26.5.1 with built-in SQLite. Its hosted counterpart is a Worker with D1 and R2. The alternate realization uses Python 3.14 with the standard-library HTTP server and SQLite. Both realizations read their own copy of the same business JSON: locally at startup, and for the Worker through build-time embedding followed by startup validation. They do not share handler code. The implementation does not include the Quarkus, FastAPI, PostgreSQL, or Podman profiles from an earlier concept document.

The wider artifact includes an eight-unit learning site, eight NotebookLM-generated videos, and database snapshots. Their existence helps distribute the method but supplies no evidence of learning effectiveness. Local requests use a fixed development workspace. The local acceptance evaluation exercises persona and tenant rules, not the hosted identity provider end to end.

4.3 Evidence and provenance

The evaluation record was completed at 2026-11-07T02:18:09Z. Its code baseline is commit `130054398f6c72c071fdeea266fcac714250fd1f`; the later artifact snapshot used for this manuscript is `233e9e46592ba48b5686dae99c7c829edb31e09c`. These historical commit IDs belong to the development repository; the public baseline is tag `artifact-v1.0.0` [17]. Eight input hashes in `content/evidence.json` identify the tested specification, handlers, runner, storage adapter, acceptance suite, and web code. Hash equality establishes identity of those bytes, not semantic correctness.

We also inspected the supplied 13-item review, the remediation record, the actual runner, and the post-deployment verification note. Historical counterexamples below are attributed to that review; we did not

rerun the obsolete implementation as a new controlled baseline for this manuscript. The report of 21 checks passing in each earlier implementation is a historical observation, not a matched experiment with the final 44 checks.

4.4 Procedure and interpretation of counts

The runner starts fresh loopback servers and databases, executes one acceptance run per language, creates one small migration fixture per direction, generates 200 maximum-field-length Unicode draft requests per source language, and imports each large pack into a fresh alternate realization. It also changes JSON limits and part values, tests unsupported startup states, and verifies a separate local media-upload directory. Owned child servers are stopped on exit.

The six small-migration results comprise three assertions for each of two fixtures: successful import, exact request-array preservation, and exact policy-history-array preservation. They are not six distinct migrations. The six capacity results similarly comprise three assertion groups for each source language: readback above 1 MiB at 200 records, rejection of record 201 without mutation, and large-pack migration with request preservation. No throughput or response-time distributions were recorded. Additional module and storage checks have different scopes and are reported separately.

5 Contract and Implementation

5.1 Representations and valid operations

Titles are limited to 120 Unicode code points and reasons to 2,000; unpaired surrogates are rejected in the validated text fields. A shared list specifies trimming characters. Timestamps use a real Gregorian date in the canonical UTC form `YYYY-MM-DDTHH:mm:ss.SSSZ`; year 0000 is rejected. Revisions are nonnegative safe-integer values no greater than 9,007,199,254,740,991. JSON Boolean values, strings, and nonintegral values are rejected. A JSON numeric spelling such as `1.0` is accepted when its parsed value is an admissible integer; the contract is about the numeric value, not that spelling.

Only a draft's creator with the designer role can correct, submit, or withdraw it. A submitted or terminal request cannot be corrected through the draft API. Correction records the preceding and resulting attributes. Withdrawal retains the request and history and counts toward the record limit. The v2 restriction applies to later approval and rejection operations without rewriting prior events.

5.2 Authority and auditing

Ordinary listing and individual request operations are tenant-scoped. A role in another tenant does not grant workspace-wide authority. Only approvers in the management tenant may export or import the authenticated owner's entire synthetic workspace or change its policy. This explicit exception preserves the workspace migration unit while defining who may exercise it. Policy changes record actor, time, and previous and resulting versions in `policyHistory`.

Imported histories are checked for known actors, tenant consistency, creator and submitter roles, allowed transitions, chronological timestamps, correction attributes, and applicable self-approval rules. This checks internal consistency under the contract. It does not authenticate the historical origin of a pack: a fabricated but contract-consistent pack can still be constructed. Signed provenance and external attestation are outside scope.

5.3 Storage and operational envelope

The contract applies joint limits of 200 retained requests, 24,000,000 request-body bytes, 16,000,000 workspace bytes, 200,000 bytes per request record, 20 events per request history, and 2,000 policy events. Decimal byte limits differ from the binary threshold of 1 MiB used in the regression. Correction reserves two history positions for submission and judgment. Capacity is therefore a conjunction of constraints, not a promise that every combination of maximum record count and maximum history size fits.

The previous local JavaScript adapter spawned Python and received the entire state through buffered standard output. The revised adapter uses SQLite in the Node.js process. The shared JavaScript repository splits large serialized workspace state into rows and updates metadata, chunks, and the revision through a conditional transaction or D1 batch. A compare-and-swap loser must not replace the winner's chunks. Learning progress uses a separate table and revision, preventing a quiz save from invalidating an unrelated business command.

These changes are engineering mechanisms supporting the envelope. The observed 200-record workloads contain newly created drafts with short histories; they do not exhaust the joint workspace or history limits. Local SQLite checks of the repository SQL do not establish every hosted D1 capacity, concurrency, or failure condition.

6 Results

6.1 RQ1 Observed conformance

Each realization passed all 44 recorded acceptance checks, with zero failures. The shared suite includes valid creation, invalid attributes and roles, creator-only submission, v2 self-approval rejection, preservation after rejection, cross-tenant access refusal, distinct-approver approval, v1 comparison behavior, stale and malformed revisions, rejection reasons, correction and withdrawal, import restrictions, and learning/business revision separation.

The representation counterexamples are part of the suite. A title containing 120 instances of U+1F600 is accepted and 121 is rejected. Non-ISO dates, an impossible calendar date, year 0000, a malformed surrogate, and an imported creator without the designer role are rejected. The result supports the exercised contract boundaries. It does not establish arbitrary Unicode normalization equivalence, grapheme-cluster limits, exhaustive parser agreement, or all possible operation sequences.

Four configuration assertions passed: one combined field-limit and new-part check per implementation, and one unsupported-state startup-rejection check per implementation. This demonstrates that selected JSON parameters affect execution. It does not show that arbitrary natural-language rule changes automatically modify business logic or that every unsupported specification is rejected.

6.2 RQ2 Preserved data in the exercised migrations

Both small directional migrations succeeded. Each source fixture creates a request with a maximum-length supplementary-plane title, corrects it, withdraws it, and changes policy. The six assertions passed, preserving the request array, including correction and withdrawal events, and the policy-history array. The changed title at export is shorter than the initial title; the initial maximum-length value remains in its creation and correction history.

For each source realization, the runner generated 200 drafts with 120-code-point titles and 2,000-code-point reasons using U+1F600. The exported data exceeded 1,048,576 bytes, remained readable, and imported into the other realization with the request array intact. Record 201 was rejected with no state change. All

six capacity assertion groups passed. The record contains a threshold assertion rather than an exact exported-byte measurement, so we do not report an invented pack size.

Five real-SQLite repository checks passed: a concurrent revision attempt succeeded once, a state above 1 MiB survived chunked roundtrip, a stale writer preserved the stored chunks, restart retained the state, and another owner could not read it. Four in-memory boundary checks and seven media checks also passed. A local media-upload check confirmed that ingestion used a separate directory and left the tracked video unchanged. Seventeen asset size/hash checks and one combined snapshot integrity and sample-identifier check passed. These heterogeneous observations are not aggregated into an overall reliability rate.

6.3 RQ3 Failures hidden by the earlier acceptance record

The supplied review describes three cross-language disagreements omitted from the earlier ASCII and conventional-date evaluation. JavaScript counted UTF-16 code units, whereas Python counted code points. JavaScript accepted a locale-style imported date that Python rejected. Python compared Boolean `true` equal to integer revision 1, whereas JavaScript rejected the update. Each discrepancy violates an exchange assumption without contradicting an earlier passing example.

The review also reports a 1,169,346-byte state containing 90 within-limit requests that exceeded the local subprocess output buffer and caused `ENOBUFS`, despite a declared 200-request limit. This is a historical review observation. Its role is to identify a hidden implementation limit; the revised workload checks demonstrate successful larger-than-1-MiB handling without measuring a general performance improvement.

Other reviewed mismatches concerned duplicated configuration, overbroad export and policy authority, missing policy auditing, absent draft correction and withdrawal, weak import-role validation, learning/business revision interference, divergent build entrypoints, misleading evidence presentation, tracked-media overwrite, snapshot identifiers, and inconsistent teaching records. These are diagnostic findings from one episode, not a representative defect taxonomy or a measured defect frequency.

6.4 Hosted observation

A separately recorded hosted check used source commit `8ef0f63b71cbbad26feb7f2d797f825d3203e50d`. The tested input hashes matched the local record. An authenticated browser loaded an existing approved request with three history events, exercised policy v2→v1→v2, and displayed two policy-audit events with actor and time while retaining the request. The hosted evidence page displayed the 44-check records and their baseline commit.

This confirms a narrow hosted read/write and presentation path after deployment. It is not a rerun of the entire acceptance suite through hosted authentication, nor a hosted 200-record capacity or bidirectional migration evaluation. The distinction is retained in the artifact and in this paper.

7 Discussion

7.1 Replaceability is broader than a passing interface suite

The case suggests that a replacement argument must address the meaning of text and time, not merely field names and status codes. It must also cover history and the operational path that transports and persists state. A specification can state a 200-record bound while an adapter imposes an earlier byte bound. An application can segregate request lists while exposing all tenants through an undocumented export path. These observations motivate checking representation, authority, and capacity as explicit replacement conditions.

The proposed method changes where decisions are documented and how their support is inspected. It does not eliminate implementation work or maintenance. A rule expressed in prose still requires operational semantics; configuration consumption only removes selected duplication. The review-and-return cycle is useful precisely because the initial specification and tests were incomplete.

7.2 What the case adds to specification driven development

Existing abstraction, contract, and model-driven approaches already justify separating business commitments from technical realization [2–8]. Contemporary SDD work directly addresses durable intent and agent governance [11–13]. Our addition is a concrete, falsifiable engineering claim: a proposed replacement should preserve specified business observations and retained history under explicit authority and workload assumptions. Export/import checks and capacity counterexamples make this claim inspectable in a stateful application.

We cannot conclude that AI caused the observed successes, that it was necessary, or that the method outperforms conventional development. A competent team using contracts and conventional tools could implement the same checks. AI is part of the construction setting and a proposed means of producing alternatives; its incremental benefit remains an empirical question.

7.3 Organizational authority remains a hypothesis

The artifact provides technical mechanisms for retaining intent, contracts, history, and evidence. Whether these mechanisms give an organization meaningful choice depends on skills, contracts, deployment access, review capacity, and migration cost. No real organization or business stakeholder participated in a measured study here. The eight teaching units do not substitute for that evidence. Business sovereignty is therefore an objective operationalized in part, not an organizational outcome established by this demonstration.

8 Threats to Validity

Construct validity. Passing selected HTTP assertions approximates contract conformance; it does not measure complete system correctness or organizational freedom. Internal consistency of imported history is weaker than authenticated provenance. The replacement gate is a proposed construct whose adequacy needs independent evaluation.

Internal validity. Specification, both implementations, and tests were revised within the same AI-assisted context. Review feedback shaped the final suite. Shared misconceptions and repair-directed evaluation can inflate apparent completeness. The earlier 21-check and final 44-check results are not a controlled before/after comparison because the evaluation instrument changed.

External validity. One small synthetic workflow, four personas, two languages, and a restricted data model do not represent enterprise systems with integration, real identity lifecycle, long-lived schema evolution, or operational incidents. SQLite is used locally in both realizations, reducing persistence diversity. The hosted checks cover only a narrow D1 path.

Conclusion validity. Assertions are dependent and heterogeneous. We report neither an estimated defect probability nor a statistical reliability claim. No effort, latency, cost, or user-outcome measurements justify superiority claims. The review's earlier counterexamples were not independently reproduced for this manuscript.

Reproducibility. The artifact pins code and input hashes and retains commands, but lacks a frozen AI generation protocol and an independently repeated run. A public replication export now provides a fixed

baseline with substituted snapshot identifiers and fresh Git history [17]. The original private history and unredacted snapshots are excluded. Public access enables replay, but does not establish independent replication or a reproducible AI generation protocol.

9 Research Agenda

A subsequent evaluation should preregister a baseline and the replacement gate, freeze a contract and withheld checks before implementation, and give independent teams no access to the alternate handler. It should include different AI providers and a non-AI comparison, record interventions and effort, and assess both initial construction and later changes.

Migration experiments should include generated operation sequences, larger and longer histories, all joint storage boundaries, malformed and fabricated packs, schema changes, identity remapping, fault injection, and write-after-cutover recovery. Hosted tests should exercise authentication and storage end to end. Field studies should measure whether business stakeholders can understand and approve rules and actually exercise an exit option. These experiments address feasibility, causal contribution, and organizational consequences separately.

10 Conclusion

This study presents Business Sovereignty Development as a specification-centered method for supporting the choice of implementation. In one constructed workflow, two language realizations passed 44 acceptance checks each, and the exercised migrations preserved request and policy histories. Review counterexamples showed why those results must be qualified by representation semantics, authority, operational limits, and evidence provenance. The findings support a bounded engineering demonstration and a research program. They do not demonstrate complete implementation independence, autonomous regeneration, or the historical inevitability of the proposed method.

Data and Software Availability

The public Business Sovereignty Replication Artifact provides source code, specifications, recorded evaluations, eight videos, and sanitized SQLite snapshots [17]. Use fixed tag artifact-v1.0.0 for the code/data baseline and `main/research/paper` for the manuscripts. The historical development commits in Section 4.3 are identified through the retained input hashes rather than being present in the fresh public history. Local reproduction requires Node.js and Python, with no cloud account or hosted credentials. The hosted demonstration remains owner-restricted. Original private histories and unredacted snapshots are not included.

AI Assistance and Authorship

AI tools assisted the construction and revision of the artifact, literature discovery, drafting, and translation of this manuscript. The retained records do not support a model-performance attribution. The human author must verify the manuscript, citations, access rights, and venue-specific disclosure and authorship requirements before submission. Affiliation, funding information, and any competing interests have not been supplied and are not inferred. This draft is not a submitted or accepted paper.

Appendix A Reproduction Procedure

Clone the public fixed baseline `artifact-v1.0.0` [17]. The development snapshot `233e9e46592ba48b5686dae99c7c829edb31e09c` is a historical provenance identifier, not a public checkout target. Confirm the eight source hashes against `content/evidence.json` before running. Node.js 22.13 or

later is required; the recorded run used 26.5.1. Python 3 is required; the recorded run used 3.14. Run from the repository root:

```
git clone --branch artifact-v1.0.0 --depth 1 https://github.com/keijijin/business-sovereignty-reproducibility.git
cd business-sovereignty-reproducibility
node scripts/build.mjs
node scripts/validate-artifact.mjs
python3 scripts/review-checks.py
```

The runner creates its own databases and loopback ports, writes a new `content/evidence.json`, and stops its child servers. Preserve the historical record separately before rerunning. Run `node scripts/check-storage.mjs`, `node scripts/check-boundaries.mjs`, `node scripts/check-media.mjs`, and `python3 scripts/verify-snapshots.py` for individually scoped checks. Do not infer a performance SLO from these commands. The hosted source uses a separate history because its source host cannot store the largest video object; business input hashes, rather than repository-wide commit equality, relate it to the evaluated artifact.

References

[1] G. M. Amdahl, G. A. Blaauw, and F. P. Brooks, Jr. (1964). Architecture of the IBM System/360. IBM Journal of Research and Development, 8(2), 87–101. Source.

[2] E. F. Codd (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377–387. Source.

[3] D. L. Parnas (1972). On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM, 15(12), 1053–1058. Source.

[4] B. Meyer (1992). Applying Design by Contract. Computer, 25(10), 40–51. Source.

[5] B. Liskov and J. M. Wing (1994). A Behavioral Notion of Subtyping. ACM Transactions on Programming Languages and Systems, 16(6), 1811–1841. Source.

[6] N. G. Leveson (2000). Intent Specifications: An Approach to Building Human-Centered Specifications. IEEE Transactions on Software Engineering, 26(1), 15–35. Source.

[7] Object Management Group (2014). Model Driven Architecture Guide Revision 2.0. OMG document ormsc/2014-06-01. Source.

[8] J. Whittle, J. Hutchinson, and M. Rouncefield (2014). The State of Practice in Model-Driven Engineering. IEEE Software, 31(3), 79–85. Source.

[9] M. Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374v2. Source.

[10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. International Conference on Learning Representations; arXiv:2310.06770v3. Source.

[11] D. B. Piskala (2026). Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants. Preprint, arXiv:2602.00180v1. Source.

[12] J. Diaz, J. Gayoso, A. Cimminio, and J. Perez (2026). Spec-Driven Development for Agentic Software Engineering: Harnessing Human–Agent Teamwork. Preprint, arXiv:2609.00252v1. Source.

[13] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux (2026). Spec-Driven Development as Prescribed and as Practiced. Preprint, arXiv:2608.25202v4. Source.

[14] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo (2015). The Oracle Problem in Software Testing: A Survey. IEEE Transactions on Software Engineering, 41(5), 507–525. Source.

[15] J. C. Knight and N. G. Leveson (1986). An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. IEEE Transactions on Software Engineering, SE-12(1), 96–109. Source.

[16] P. Runeson and M. Höst (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. Empirical Software Engineering, 14(2), 131–164. Source.

[17] K. Jin (2026). Business Sovereignty Replication Artifact, version 1.0.0. Source, specifications, evaluations, videos, and sanitized database snapshots. Fixed baseline.